

טריקים ושטיקים למידת מכונה

5 באוגוסט 2026

גיא יער-און

נגזרות, גרדיאנטים וכו' (הקדמה מתמטית)

* לשים לב כי אם S הוא סקלר, נוכל לכתוב: $S^2 = SS^T$, שכן הפעלת טרנספוז על סקלר משאירה אותו כמו שהוא.
* נסמן A כמטריצה ו- w, x כווקטורים. אזי:

$$1. \nabla(w^T x) = x$$

$$2. \nabla(w^T A w) = (A + A^T)w$$

$$3. \nabla(w^T w) = 2w \text{ אזי } A = I \text{ כאשר } 2 \text{ במקרה פרטי של}$$

$$4. \text{ כלל חשוב: } \nabla f(g(w)) = \frac{\partial f}{\partial g} \cdot \nabla(g)$$

$$5. \frac{\partial(Aw)}{\partial w} = A$$

$$6. \text{ סיגמואיד: } \sigma(z) = \frac{1}{1+e^{-z}}. \text{ מתקיים: } \frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

$$7. \frac{\partial \ln(w^T x)}{\partial w} = \frac{1}{w^T x} \cdot x$$

$$8. \nabla \text{tr}(AB) = B^T$$

$$9. \text{Bias} = \mu - \mathbb{E}[w]$$

קרנלים

טיפ טוב, למצוא מיפוי לדברים כמו $K(x, x') = \min(x, x')$. אנחנו מתרגמים לביטוי לינארי כזה:

$$\min(x, x') = \frac{x + x' - |x - x'|}{2}$$

ומשם קל יותר להתקדם.

K-means

* כאשר עובדים עם מרחק L_1 האלגו' מעודכן להיות לפי החציון.

SVM ואופטימיזציה

1. כאשר רוצים בהינתן פתרון לבעיה מקורית w^* להוכיח שפתרון $f(w^*)$ הוא אופטימלי לבעיה שעברה שינוי - מוכיחים שהאילוץ עם $f(w^*)$ נשמרו ושכן $f(w^*)$ אופטימלי (בשליה, מניחים שלא ואז מגיעים לכך ש- w^* אינו אופטימלי). אופציה אחרת היא להוכיח אי שוויון משני הצדדים.
2. הפתרון הדואלי של SVM:

$$W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (x_i^T x_j)$$

3. הפתרון של Soft SVM:

$$\min_{w,b} \left[\|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w^T x_i - b)) \right]$$

4. כאשר אנו עובדים עם פתרון של Hard-SVM w^* הוא בהכרח מהווה (עם פונקציית המטרה שלו) חסם לערך הפתרון של Soft-SVM.
5. SVM מחולק לזוג חלקים:
 - אימון: מציאת המשקולות. פותרים את בעיית האופטימיזציה לעיל.

- חיזוי: מקבלים x ורוצים את התיוג שלו. (מעל או מתחת לקו). זה בעצם:

$$\hat{y} = \text{sign} \left(w^T \phi(x_i) \right) = \left(\sum_{i=1}^n \alpha_i y_i K(x, x_i) \right)$$

אלגוריתמים שיש לזכור בעל פה

אלגוריתם הפרספטרון

Algorithm 1 Perceptron Algorithm

```

Initialize  $w$  randomly
while not all samples are classified correctly do
  for  $i = 1$  to  $n$  do
    Predict  $\hat{y}_i = \text{sign}(w^T x_i)$ 
    if  $y_i \neq \hat{y}_i$  then
      if  $y_i > 0$  then
         $w \leftarrow w + x_i$ 
      else
         $w \leftarrow w - x_i$ 
      end if
    end if
  end for
end while

```

אלגוריתם הפרספטרון כאשר התוויות הן $\{0, 1\}$

Algorithm 2 Perceptron Training with $\{0, 1\}$ Labels

Require: Training set $\{(x_i, y_i)\}_{i=1}^n$ with $y_i \in \{0, 1\}$, learning rate η

Initialize the weight vector w (e.g. $w \leftarrow 0$)

repeat

for each example (x_i, y_i) in the dataset **do**

 Compute the prediction:

$$\hat{y}_i \leftarrow \begin{cases} 1 & \text{if } w^\top x_i \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

 Update: $w \leftarrow w + \eta (y_i - \hat{y}_i) x_i$

end for

until no example is misclassified (or a maximum number of epochs is reached)

return w

אלגוריתם לאימון ניורון בודד באמצעות SGD ו Hinge Loss

Algorithm 3 Single Neuron Training via SGD with Hinge Loss

Require: Training set $\{(x_i, y_i)\}$, learning rate η
 Initialize the weight vector w randomly
repeat
 Sample a random example (x_i, y_i) from the dataset
 if the classification is correct (positive margin) **then**
 Do nothing
 else
 Update: $w \leftarrow w + \eta y_i x_i$
 end if
until the model stops improving

כמו הקודם, רק עם שולי בטחון

Algorithm 4 Single Neuron Training via SGD with Hinge Loss

Require: Training set $\{(x_i, y_i)\}$, learning rate η
 Initialize the weight vector w randomly
repeat
 Sample a random example (x_i, y_i) from the dataset
 Compute the (sub)gradient of the hinge loss:

$$g \leftarrow \begin{cases} -y_i x_i & \text{if } y_i w^\top x_i < 1 \\ 0 & \text{otherwise} \end{cases}$$

Update: $w \leftarrow w - \eta g$
until the model stops improving

רגרסיה לינארית עם גרדיאנט דיסנט

Algorithm 5 Linear Regression Training via Gradient Descent

Require: Training set $\{(x_i, y_i)\}_{i=1}^n$, learning rate η
 Initialize the weight vector w^0 randomly
repeat
 Predict $\hat{y}_w(x_i)$ for the training examples
 Compute the gradient: $\frac{\partial \ell}{\partial w} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_w(x_i)) (-x_i)$
 Update: $w^{t+1} \leftarrow w^t - \eta \nabla_w \ell$
until the model stops improving

רגרסיה לוגיסטית בינארית

SGD -

Algorithm 6 Binary Logistic Regression Training via SGD

Require: Training set $\{(x_i, y_i)\}$ with $y_i \in \{0, 1\}$, learning rate η
 Initialize the weight vector w randomly
repeat
 Sample a random example (x_i, y_i) from the dataset
 Compute the prediction: $\hat{y}_i \leftarrow \sigma(w^\top x_i) = \frac{1}{1 + e^{-w^\top x_i}}$
 Compute the gradient of the log loss: $g \leftarrow (\hat{y}_i - y_i) x_i$
 Update: $w \leftarrow w - \eta g$
until the model stops improving

- רגיל

Algorithm 7 Binary Logistic Regression Training via Gradient Ascent

Require: Training set $\{(x_i, y_i)\}_{i=1}^n$ with $y_i \in \{0, 1\}$, learning rate η
 Initialize the weight vector w randomly
repeat
 Update: $w \leftarrow w + \eta \sum_{i=1}^n (y_i - \sigma(w^\top x_i)) x_i$
until the model stops improving

Softmax regression (סיווג כאשר מס' המחלקות $k \geq 3$)**Algorithm 8** Softmax Regression Training

Require: Training set $\{(x_i, y_i)\}$, number of classes K , learning rate η
 Initialize all weights in the matrix W with random values
repeat
 for each training example (x_i, y_i) **do**
 Predict: compute scores for all K classes: $z = W^T x_i$
 Convert scores to probabilities using softmax, obtaining a vector p
 Compute the cross-entropy loss (for monitoring: verify the model improves and the loss decreases; stop training once the loss is sufficiently low)
 for each class $c = 1$ **to** K **do**
 Compute the gradient: $(p_c - \text{target}) x_i$
 Update the class weight vector: $w_c \leftarrow w_c - \eta (p_c - \text{target}) x_i$
 end for
 end for
until converged

אלגוריתם חמדת למציאת עץ החלטה

נתבונן בפסאודו הבא:

:BuildTree(Date, Features)

א. יצירת שורש: יוצרים צומת Root.

ב. אם כל הדגימות בדטה הן Yes (כולן חיוביות) תחזיר את השורש עם התווית Yes.

ג. אם כל הדגימות בדטה הן No (כולן שליליות) תחזיר את השורש עם התווית No.

ד. אם אין יותר פיצ'רים לבדיקה אך הדטה עדיין מעורבב - החזר את השורש עם התווית הנפוצה ביותר בדטה. אחרת:

1. בחירת הפיצ'ר הטוב ביותר: בוחרים את הפיצ'ר A שמפצל את הדטה בצורה הכי טהורה. (בהמשך נדון כיצד).2. יצירת ענפים: לכל ערך אפשרי v של A (למשל: sunny, cloudy, rainy וכו')

• יוצרים ענף חדש מתחת לשורש.

• מגדירים תת קבוצה של דטה $Data_v$ שכוללת רק את הדגימות שבהן הפיצ'ר $A = v$.

• אם תת הקבוצה ריקה: מוסיפים עלה עם התווית הנפוצה ביותר בדטה המקורי.

• אחרת: קוראים ברקורסיה לאלגוריתם שוב עבור תת הקבוצה והפיצ'רים שנותרו (ללא A). כלומר BuildTree(Data_v, Features \ {A}).

ו. בסוף הרקורסיה: העץ מוכן.

הערה 1. להלן פסודו פורמלי יותר:

Algorithm 9 Decision Tree Construction via BuildTree(Data, Features)

Require: Training data $Data$, feature set $Features$

Create a root node $Root$

if all examples in $Data$ are positive (Yes) **then**

return $Root$ with label Yes

end if

if all examples in $Data$ are negative (No) **then**

return $Root$ with label No

end if

if $Features$ is empty **then**

return $Root$ with the most common label in $Data$

end if

Choose the best feature A that splits $Data$ most purely

for all possible values v of A (e.g., sunny, cloudy, rainy) **do**

 Create a new branch below $Root$ for $A = v$

 Let $Data_v \subseteq Data$ be the subset of examples where $A = v$

if $Data_v$ is empty **then**

 Add a leaf with the most common label in $Data$

else

 Recursively call BuildTree($Data_v$, $Features \setminus \{A\}$)

end if

end for

return $Root$

נוסחאות קשורות שלא מופיעות בדף נוסחאות:
- אנתרופיה:

$$H(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

- בוחרים בפיצ'ר A הממקסם את $IS(S, A)$:

$$IS(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

אלגוריתם $GDBOOST$ (Gradient Boosting)

- אתחול: קבע לכל דגימה i את $\hat{y}_i^{(0)} = 0$ ולכן $f_1(x) = 0$.
- לכל $1 \leq m \leq M$:
 - חשב את השארית עבור הדגימה: $r_i = \hat{y}_i^{(m-1)} - y_i$ (הניבוי עבור הערך הקודם, פחות הערך האמיתי)
 - אמן עץ חדש h_m על פני מערך נתוני האימון החדש: $\{(x_i, r_i)\}_{i=1}^n$
 - עדכן את הפונקציה בתוספת קבוע α קטן: $f_m(x) = f_{m-1}(x) + \alpha h_m$ (משמש כקצב למידה, ומחליף את γ_m שקודם ראינו).
 - חשב ניבויים מעודכנים לכל הדגימות באמצעות הפונקציה החדשה - $\hat{y}_i^{(m)} = f_m(x_i)$
 - לאחר סיום ריצת הלולאה, מתקיים לבסוף $f_M(x) = \sum_{m=1}^M \alpha h_m(x)$.

אלגוריתם Naive Bayes

בהינתן וקטור קלטים $\{x_1, \dots, x_n\}$ ואוסף מחלקות $\{c_1, \dots, c_k\}$ רוצים לבחור לאיזו קבוצה הכי שייך. תחת הנחת אי תלות:

$$c^* = \arg \max_{i=1, \dots, k} p(c_i) \prod_{j=1}^d p(x_j | c_i)$$

Algorithm 10 Naive Bayes — Training and Prediction

Require: Training set $\{(x_i, y_i)\}_{i=1}^n$ where $x_i = (x_i^{(1)}, \dots, x_i^{(d)})$ and $y_i \in \{1, \dots, C\}$

— **Training (parameter estimation by counting)** —

for each class $c \in \{1, \dots, C\}$ **do**

Estimate the prior:

$$P(y = c) \leftarrow \frac{\#\{i : y_i = c\}}{n}$$

for each feature $j \in \{1, \dots, d\}$ and each value v **do**

Estimate the conditional likelihood:

$$P(x^{(j)} = v \mid y = c) \leftarrow \frac{\#\{i : x_i^{(j)} = v \wedge y_i = c\}}{\#\{i : y_i = c\}}$$

end for

end for

— **Prediction for a new point** $x = (x^{(1)}, \dots, x^{(d)})$ —

Using the naive (conditional independence) assumption, predict:

$$\hat{y} \leftarrow \arg \max_{c \in \{1, \dots, C\}} P(y = c) \prod_{j=1}^d P(x^{(j)} = x^{(j)} \mid y = c)$$

return $\hat{y}$

אלגוריתם KNN

הערה. אלגוריתם זה מניח דטה קיים מסווג, ובהינתן דוגמה חדשה הוא מסווג. אין כאן שלב אימון.

Algorithm 11 KNN (K Nearest Neighbors)

Require: metric d (distance function), number of neighbors k , test sample x

Find the k training examples closest to x according to the metric d

return the majority class among the k nearest neighbors of x

אלגוריתם K-means

א. אתחול - בוחרים באופן אקראי K אבות טיפוס ראשוניים מתוך המרחב

$$\mu^0 = (\mu_1^0, \dots, \mu_k^0)$$

ב. נרוץ בלולאה עבור $t = 1, 2, \dots$ (עד להתכנסות למינימום מקומי)

1. E-Step: כל דגימה x_i משויכת לאשכול האב טיפוס הקרוב ביותר אליה בהתבסס על אבות הטיפוס מהאיטרציה הקודמת. כלומר:

$$\text{Cluster}^{(t-1)}(x_i) = \arg \min_j \|x_i - \mu_j^{t-1}\|_2^2$$

2. M-step: מחשבים מחדש את המיקום של כל אב טיפוס בתור הממוצע (Mean) של כל הדגימות ששייכו לאותו אשכול בצעד הקודם. כלומר: - נגדיר לכל אשכול $1 \leq j \leq k$ את אוסף כל הנקודות שמשויכות לאשכול זה:

$$c_j = \{x_i \mid \text{Cluster}^{(t-1)}(x_i) = j\}$$

- נעדכן את האב טיפוס החדש μ_j^t (הממוצע) של האשכול j באיטרציה הנוכחית:

$$\mu_j^t = \frac{1}{|c_j|} \sum_{x \in c_j} x$$

האלגוריתם יעצור רק כאשר אין יותר שינוי במיקומי אבות הטיפוס ובשייכוים של הנקודות לאשכולות בין איטרציה אחת לבאה אחריה. (כלומר

אם מתקיים לכל $1 \leq j \leq k$ כי $c_j^{(t-1)} = c_j^{(t)}$ נעצור). מדוע? אם הקבוצות לא זזו - גם כשנפעיל את ה- $step$ M לחשב את המרכזים החדשים - הממוצעים של כל קבוצה ישארו בדיוק אותו דבר.

Algorithm 12 Principal Component Analysis (PCA)**Require:** Data points $\{x_i\}_{i=1}^n \subseteq \mathbb{R}^D$, target dimension $d < D$ Compute the mean: $\mu \leftarrow \frac{1}{n} \sum_{i=1}^n x_i$ Center the data: $\tilde{x}_i \leftarrow x_i - \mu$ for all i Compute the covariance matrix: $\Sigma \leftarrow \frac{1}{n} \sum_{i=1}^n \tilde{x}_i \tilde{x}_i^\top$ Compute the eigendecomposition of Σ Let $U \in \mathbb{R}^{D \times d}$ be the matrix whose columns are the d eigenvectors corresponding to the d largest eigenvaluesProject the data: $z_i \leftarrow U^\top \tilde{x}_i$ for all i **return** Low-dimensional representations $\{z_i\}_{i=1}^n \subseteq \mathbb{R}^d$ **אלגוריתם תערובת הגאוסיאנים**

פונקציית המטרה אינה קמורה ולא ניתנת לפתרון אנליטי ישיר. לכן משתמשים בשיטה דומה ל-GD. בכל איטרציה ישנם שני שלבים מרכזיים:
 1. E-Step: הפרמטרים של הגאוסיאן קבועים. כלומר $\theta^{(t)} = (\pi^{(t)}, \mu^{(t)}, \Sigma^{(t)})$. המטרה היא לחשב לכל נקודה x_i ואשכול k את ה"אחריות" שתחושב:

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}$$

למעשה זה בדיוק ההסתברות הפוסטרירורית - בהינתן שראינו את x_i מה הסיכוי שרכיב הגאוסיאן ה- k יצר אותה.
 2. M-Step: בשלב זה האחריות γ_{ik} שחישבנו בשלב הקודם מקובעות. מעדכנים כעת את פרמטרי המודל $\theta^{(t+1)}$ על מנת למקסם את הנראות:
 א. מעדכנים את הממוצעים

$$\mu_k^{(t+1)} = \frac{\sum_{i=1}^n \gamma_{ik} x_i}{\sum_{i=1}^n \gamma_{ik}}$$

כלומר הממוצע של הגאוסיאן ה- k (המחלקה שלו) יהיה הממוצע המשוקלל של כל הנקודות בדטה.
 ב. עדכון מטריצת השונות המשותפת:

$$\Sigma_k^{(t+1)} = \frac{\sum_{i=1}^n \gamma_{ik} (x_i - \mu_k^{(t+1)}) (x_i - \mu_k^{(t+1)})^T}{\sum_{i=1}^n \gamma_{ik}}$$

ג. עדכון משקלי התערובת -

$$\pi_k^{(t+1)} = \frac{1}{n} \sum_{i=1}^n \gamma_{ik}$$

המשקל האפריורי החדש של האשכול הוא פשוט סך כל ה"אחריות" שרכיב k לקח על עצמו מתוך n הנקודות הקיימות.
 האלגוריתם חוזר חלילה על שלבים אלו - עד שפרמטרי המודל או ערך הלוג-נראות מפסיקים להשתנות.

Algorithm 13 Gaussian Mixture Model Training via EM (probabilistic notation)**Require:** Data points $\{x_i\}_{i=1}^n$, number of clusters K Initialize for every cluster k : the prior $P(z = k)$, the mean μ_k , and the covariance Σ_k **repeat****E-Step:** for every point x_i and cluster k , compute the posterior probability that cluster k generated x_i :

$$P(z_i = k | x_i) = \frac{P(z_i = k) \cdot P(x_i | z_i = k)}{\sum_{j=1}^K P(z_i = j) \cdot P(x_i | z_i = j)}$$

where $P(x_i | z_i = k) = \mathcal{N}(x_i | \mu_k, \Sigma_k)$ **M-Step:** fix the posteriors $P(z_i = k | x_i)$ and update every cluster k :

Update the mean — a weighted average of all points, weighted by the posteriors:

$$\mu_k \leftarrow \frac{\sum_{i=1}^n P(z_i = k | x_i) x_i}{\sum_{i=1}^n P(z_i = k | x_i)}$$

Update the covariance — a weighted spread around the new mean:

$$\Sigma_k \leftarrow \frac{\sum_{i=1}^n P(z_i = k | x_i) (x_i - \mu_k)(x_i - \mu_k)^T}{\sum_{i=1}^n P(z_i = k | x_i)}$$

Update the prior — the average responsibility cluster k took over the data:

$$P(z = k) \leftarrow \frac{1}{n} \sum_{i=1}^n P(z_i = k | x_i)$$

until the parameters or the log-likelihood stop changing**return** $P(z = k)$, μ_k , Σ_k for all clusters k **Diffusion models**

• שלב האימון: אנו רוצים לאמן רשת ניורונים ϵ_θ שתדע לחזות את הרעש שהוספנו לתמונה. אנו בוחרים תמונה מקורית $x_0 \sim q(x_0)$, דוגמים זמן צעד אקראי $t \sim U[1, T]$ ומייצרים רעש גאוסיאני $\epsilon \sim \mathcal{N}(0, 1)$. בשורה 5 - אנו מחשבים את המרחק הריבועי (MSE) בין הרעש האמיתי לבין הרעש שהמודל שלנו חזה. אנו מנסים למזער הפרש זה ולכן לוקחים גרדיאנט:

$$\nabla_\theta \|\epsilon - \epsilon_\theta(\sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon, t)\|^2$$

עושים תהליך זה באופן איטרטיבי עד להתכנסות.

Algorithm 14 Training**repeat** $x_0 \sim q(x_0)$ $t \sim \text{Uniform}(\{1, \dots, T\})$ $\epsilon \sim \mathcal{N}(0, \mathbf{I})$

Take gradient descent step on

$$\nabla_\theta \|\epsilon - \epsilon_\theta(\sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon, t)\|^2$$

until converged

• שלב 2: תהליך היצירה. כאן משתמשים במודל המאומן על מנת ליצור תמונה חדשה מאפס. מתחילים עם רעש טהור לחלוטין $x_T \sim \mathcal{N}(0, I)$ ומבצעים לכל $1 \leq t \leq T$ את הצעד ההפוך. מחשבים את הצעד אחורה באמצעות הנוסחה:

$$z_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z$$

באשר:

- $\epsilon_\theta(x_t, t)$: ניחוש המודל לפי רשת הניורונים שבנינו.- α הם היפר פרמטרים שנקבעו מראש, קובעים כמה חזק כל רעש.- z הוא רעש אקראי שאנו מוסיפים על מנת להכניס אקראיות ונורמליות.- σ_t מקדם שקובע כמה רעש אקראי יש להוסיף חזרה בצעד הנוכחי.- x_0 : למעשה חוזרים על תהליך זה עד שמגיעים חזרה לתמונה המקורית.

Algorithm 15 Sampling

```

 $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
for  $t = T, \dots, 1$  do
   $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
   $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
end for
return  $\mathbf{x}_0$ 

```

פרספטרום עם קרנל טריק

שים לב כי שלב העדכון של הפרספטרום הוא למעשה שאנו מחשבים את הסימן $\text{sign}(w^T x_i)$.
 כעת נרצה לחשב את

$$\text{sign}(w^T \phi(x_i))$$

נשים לב כי את w ניתן לכתוב כ"ל של נקודות האימון מההטלה. תמיד הוא כזה. לכן $w_t = \sum_i \alpha_i \phi(x_i)$. משם:

$$\text{sign}(w_t^T \phi(x_i)) = \text{sign} \left(\left(\sum_j \alpha_j \phi(x_j) \right)^T \phi(x_i) \right) = \text{sign} \left(\sum_j \alpha_j \phi(x_j)^T \phi(x_i) \right) = \text{sign} \left(\sum_j \alpha_j K(x_j, x_i) \right)$$

בסיום, אם נבצע $w = w + y_i \phi(x_i)$ בעדכון נשבור את הקרנל טריק. לכן, נשים לב כי עדכון $w = w + y_i \phi(x_i)$ הוא בסך הכל להוסיף את y_i (או 1 או -1) למקדם של ה- $\phi(x_i)$. לכן:
 נציג את הפסודוקוד הבא:

Preceptron ($\{(x_i, y_i)\}_{i=1}^T$)
 א. אתחל את $\alpha_1, \dots, \alpha_T$ באפסים.
 ב. עד שהדגימות לא מסווגות נכון כולן:
 - לכל $1 \leq i \leq n$
 - חזה $\hat{y}_i = \text{sign} \left(\sum_j \alpha_j K(x_j, x_i) \right)$
 - אם $y_i \neq \hat{y}_i$ בצע $\alpha_i = \alpha_i + y_i$

Word Embeddings**Algorithm 16** Skip-gram (word2vec) Training via SGD

Require: Text corpus $w_1, \dots, w_T$, vocabulary V , window size m , embedding dimension d , learning rate η
 Initialize the parameter vector $\theta \in \mathbb{R}^{2|V|d}$ randomly, containing for each word $w \in V$ a center vector $v_w \in \mathbb{R}^d$ and a context vector $u_w \in \mathbb{R}^d$

repeat

Define the conditional probability of a context word o given a center word c via Softmax:

$$\Pr[o | c] = \frac{\exp(u_o^\top v_c)}{\sum_{w \in V} \exp(u_w^\top v_c)}$$

Compute the objective (negative average log-likelihood):

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log \Pr(w_{t+j} | w_t; \theta)$$

Compute the gradient $\nabla_\theta J(\theta)$

Update: $\theta \leftarrow \theta - \eta \nabla_\theta J(\theta)$

until the model stops improving

return Word vectors $\{(v_w, u_w)\}_{w \in V}$

Transformers

פסאודו רעיוני. מה שמסומן ב* זו שכבה שלא תמיד מופיעה.

Algorithm 17 Transformer (Encoder–Decoder) — Full Pipeline

Require: Input text, vocabulary of sub-word tokens (e.g. built via BPE), embedding dimension d_{model} , number of heads h , number of blocks N

— **Tokenization & Embedding** —

Split the input text into sub-word tokens $t_1, \dots, t_n$ (e.g. using BPE)

Convert each token to a vector via a learned embedding lookup: $x_i \in \mathbb{R}^{d_{model}}$

Add positional information: $x_i \leftarrow x_i + PE_i$ {e.g. fixed sine/cosine encoding}

Stack the vectors into a matrix $X \in \mathbb{R}^{n \times d_{model}}$

— **Encoder** —

for $\ell = 1$ **to** N encoder blocks **do**

Multi-Head Self-Attention: for each head $j \in \{1, \dots, h\}$ with $d_k = d_{model}/h$:

$$Q = XW_Q^{(j)}, \quad K = XW_K^{(j)}, \quad V = XW_V^{(j)}$$

$$\text{head}_j = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Concatenate all heads: $A \leftarrow \text{Concat}(\text{head}_1, \dots, \text{head}_h)$

Add & Norm: $X \leftarrow \text{LayerNorm}(X + A)$

Feed-Forward (applied to each token independently):

$$\text{FFN}(x) = \max\{0, xW_1 + b_1\}W_2 + b_2$$

Add & Norm: $X \leftarrow \text{LayerNorm}(X + \text{FFN}(X))$

end for

Encoder output: context-dependent representations $z_1, \dots, z_n$

— **Decoder*** — {*only in encoder–decoder models, e.g. machine translation; encoder-only models (classification, tagging) skip this}

Initialize the output sequence with a special start token

for $\ell = 1$ **to** N decoder blocks **do**

Masked Self-Attention: as above, but each position may only attend to previous positions $\{y_j\}_{j=1}^{i-1}$

 Add & Norm

Cross-Attention*: queries come from the decoder, keys and values from the encoder outputs $\{z_i\}_{i=1}^n$
 {*absent in decoder-only models, e.g. GPT-style}

 Add & Norm

 Feed-Forward, then Add & Norm

end for

— **Output** —

Apply a linear layer followed by softmax to obtain a probability distribution over the vocabulary

— **Inference (autoregressive generation)** —

repeat

 Select the most probable next token from the distribution

 Append it to the generated sequence and feed the sequence back into the model

until an end-of-sequence token is produced

הוכחת חסם ההתכנסות של הפרספטרון

הערה. זה די חשוב. לא למדנו את זה אבל זה יכול להופיע בהמון שאלות.

נתונים $\{(x_i, y_i)\}_{i=1}^n$ נתונים כך ש $x_i \in \mathbb{R}^d, y_i \in \{+1, -1\}$. יהי M מס' הטעויות הכולל שמבצע הפרספטרון.

יהי $R = \max_i \|x_i\|$. וכן, נניח שקיים u כך ש $\|u\| = 1$ כך ש $y_i < u, x_i \geq \gamma$ (משמעות הדבר היא כי קיים כיוון כלשהו מפריד -

כלומר אפשר להפריד באמצעותו את הנק' החיוביות והשליליות, וגם מהוקטור הזה הנקודות נמצאות בטווח מינימלי מסויים ממנו של γ (שוליים). כלומר, הנתונים ניתנים להפרדה לינארית קשיחה. ההנחה הזו נתונה תמיד כאשר הבעיה נתונה להפרדה לינארית!

טענה:

$$M \leq \frac{R^2}{\gamma^2}$$

הוכחה:

. נניח שטעינו על (x_i, y_i) בעדכון k . אזי,

$$\langle w_k, u \rangle = \langle w^{k-1} + y_i x_i, u \rangle = \langle w^{k-1}, u \rangle + y_i \langle x_i, u \rangle \geq \langle w^{k-1}, u \rangle + \gamma$$

לאחר M עדכונים, נקבל כי $\langle w_k, u \rangle \geq M\gamma$ (באינדוקציה).
כעת,

$$\|w^k\|^2 = \|w^{k-1} + y_i x_i\|^2 = \|w^{k-1}\|^2 + 2y_i \langle w^{k-1}, x_i \rangle + \|x_i\|^2 \leq \|w^{k-1}\|^2 + 0 + R^2$$

שהרי $2y_i \langle w^{k-1}, x_i \rangle \leq 0$ שכן טעות בפרספטרום קוראת כאשר $y_i(w^{k-1})^T x_i \leq 0$
כעת, באינדוקציה שוב מקבלים $\|w^k\|^2 \leq MR^2$
כעת, מאי שוויון קושי שוורץ מקבלים את הדרוש:

$$M\gamma \leq \langle w^M, u \rangle \leq \|w^M\| \|u\| \leq \sqrt{MR}$$

לכן:

$$M \leq \frac{R^2}{\gamma^2}$$

PAC - כללים לפתרון

הגדרה 2. תהי $\mathcal{H}$ מחלקת היפותזות. $\mathcal{H}$ היא PAC-learnable אם קיים אלגוריתם A שמוצא השערה $h = A(S_n)$ עבור מדגם S_n כך שמתקיים:
• לכל $\varepsilon, \delta > 0$ קיים $N_{\varepsilon, \delta}$ (אשר חייב להיות פולינומי ב $\frac{1}{\delta}, \frac{1}{\varepsilon}$) כך שלכל $n > N_{\varepsilon, \delta}$ ולכל התפלגות $\mathcal{D}$ יתקיים:

$$P(\text{Err}_{\mathcal{D}}(A(S_n)) < \varepsilon) > 1 - \delta$$

כאשר ההסתברות מעל מרחבי S_n האפשריים.

נקבל תרגיל - תהי מחלקת היפותזות H כלשהי. הוכיחו שהיא למידת PAC במקרה הריאזיבילי.
א. עלינו להגדיר אלגוריתם מסוג ERM (לרוב טריוואלי) למציאת הפרמטרים a, b וכל המודל והחזרתם.

ב. עלינו להוכיח $\Pr[\text{Err}_{\mathcal{D}}(A(S)) > \varepsilon] \leq \delta$. לשם כך:

1. נתבונן על המקרים השונים שיכולים להוביל לשגיאה, ונחסום כל אחד מהם בנפרד.

2. לרוב, אם למשל יש לנו 2 מקומות לטעות במאורעות A_1, A_2 נוכל לחסום

$$\Pr[\text{Err}_{\mathcal{D}}(A(S)) > \varepsilon] \leq \Pr[A_1 \geq \frac{\varepsilon}{2}] + \Pr[A_2 \geq \frac{\varepsilon}{2}]$$

3. נעזר במשפט ערך הביניים ונגדיר פונקציה, ולפי ערך הביניים נוכיח כי קיים a^* כך ש $f(a, a^*) = \frac{\varepsilon}{2}$.

4. כתוצאה מסעיף 3, נוכל לבצע חישובים מהצורה: שכן, הסיכוי שעברנו את $\frac{\varepsilon}{2}$ זה שהגרלנו x_i המקיים $x_i > a^*$ (שכן שם ההסתברות היא חצי אפסילון בדיוק), ולכן $x_i > a$. כלומר, ההסתברות שנהיה גדולים מ $\frac{\varepsilon}{2}$ היא אם אף אחת מהנקודות לא נפלה ב (a, a^*) . משם אפשר להמשיך ולחסום. הרעיון דומה ברוב השאלות.

$$\Pr[A_1 \geq \frac{\varepsilon}{2}] \leq \Pr[\forall i \in [n] : x_i \notin (a, a^*)] = \prod_{i=1}^n \Pr[x_i \notin (a, a^*)] =$$

$$\prod_{i=1}^n (1 - \frac{\varepsilon}{2}) = (1 - \frac{\varepsilon}{2})^n$$

5. משם, מצליחים לחסום את המאורע ולדרוש שזה יהיה לכל היותר δ ומקבלים את הדרוש.

נתבונן בתרגיל הבא להדגמת השלבים הנ"ל:

יהי מרחב $\mathbb{R}^d$. נגדיר לכל זוג $0 \leq a \leq b$ ממשיים את ההיפותזה $h_{a,b} : \mathbb{R}^d \rightarrow \{0, 1\}$ המוגדרת באופן הבא:

$$h_{a,b}(x) = \begin{cases} 1 & a \leq \|x\|_2 \leq b \\ 0 & \text{o.w.} \end{cases}$$

כלומר, נקודה מסווגת כ-1 אם היא נמצאת בתוך טבעת בין הרדיוסים a ו- b . תהי מחלקת ההיפותוזות:

$$\mathcal{H} = \{h_{a,b} | 0 \leq a \leq b\}$$

הוכיחו ישירות כי $\mathcal{H}$ היא למידת PAC במקרה הריאזיבילי. הניחו כי התפלגות השוליים של x רציפה.

פתרון:

תחילה, נגדיר את אלגוריתם ה-ERM הבא:

יהי מדגם נקודות מסווגות $\{(x_i, y_i)\}_{i=1}^n$ כך ש- $y_i \in \{0, 1\}$. נגדיר את האלגוריתם באופן הבא:

1. חשב את הקבוצה

$$Y_1 = \{x_i | i \in [n] \wedge y_i = 1\}$$

אם $Y_1 = \emptyset$ החזר $(0, 0)$.

2. הגדר $\hat{b} = \operatorname{argmax}_{x \in Y_1} \{\|x\|_2\}$

3. הגדר $\hat{a} = \operatorname{argmin}_{x \in Y_1} \{\|x\|_2\}$

4. החזר את $(\hat{a}, \hat{b})$.

בבירור, האלגוריתם אכן מחזיר בהכרח מקטע $[\hat{a}, \hat{b}] \subseteq [a, b]$ שכן $[a, b]$ קיימים מהנתון על הריאזיביליות. וכן, אם נניח בשלילה כי $\hat{a} < a$ זה גורר כי קיימת נקודה במדגם (x_i, y_i) שסווגה כ-1 ($y_i = 1$) אך לא נמצאת באזורי הטבעת האמיתיים (a) - בסתירה. הסתירה מהכיוון השני על ההנחה כי $\hat{b} > b$ זהה. מכאן, בהכרח המקטע שלנו מוכל בתוך מקטע קיים $[a, b]$.
כעת, נרצה להוכיח את הדבר הבא: לכל $\varepsilon > 0$ $\delta > 0$:

$$Pr[Err_{\mathcal{D}}(A(S)) > \varepsilon] \leq \delta$$

לשם כך, נבחין שהשגיאה של האלגוריתם שלנו מגיעה משני אזורים בלבד.

הרי, האלגוריתם מחזיר מקטע שמוכל בתוך המקטע האמיתי. לכן, השגיאה של האלגוריתם מגיעה ע"י:

1. חישוב $\hat{a}$ המקיים $a < \hat{a}$ ולכן מפספסים חלק מהמקטע. נקרא לטעות זו טעות מסוג 1 ונסמן ע"י המאורע A_1

2. חישוב $\hat{b}$ המקיים $\hat{b} < b$ ולכן מפספסים מלמעלה חלק מהמקטע. נקרא לטעות זו טעות מסוג 2 ונסמן ע"י המאורע A_2 .
אם כן,

$$Pr[Err_{\mathcal{D}}(A(S)) > \varepsilon] \leq Pr[A_1 \geq \frac{\varepsilon}{2}] + Pr[A_2 \geq \frac{\varepsilon}{2}]$$

מדוע? נבחין כי שגיאה מתרחשת כאשר קורה מאורע A_1 או מאורע A_2 בלבד. אם נניח בשלילה כי ההסתברות לשניהם קטנה מחצי אפסילון - נקבל שהכוללת קטנה מאפסילון בסתירה. מכאן, ההסתברות שהאלגוריתם יטעה היא לכל היותר חסומה בסכום ההסתברויות לפי A_1 ו- A_2 טועים ביותר מחצי אפסילון.

כעת, נחסום את $Pr[A_1 \geq \frac{\varepsilon}{2}]$. אם כן,

$$Pr[x \in (a, a^*)] = \frac{\varepsilon}{2} \text{ עבור } a^* \text{ עבורו } Pr[x \in (a, a^*)] = \frac{\varepsilon}{2}$$

נגדיר את הפונקציה הבאה: $f(t) = Pr[a \leq \|x\|_2 \leq t]$. מהנתון, היא רציפה כי ההתפלגות השולית של x רציפה. כמו כן: $f(a) = 0$. וכן, נראה כי: $f(b) > \frac{\varepsilon}{2}$ לכן בהכרח קיים לפי ערך הביניים a^* הקטע שבו $Pr[x \in (a, a^*)] = \frac{\varepsilon}{2}$. באופן דומה בה"כ נאמר כי קיים b^* שמקיים: $Pr[x \in (b^*, b)] = \frac{\varepsilon}{2}$.

כעת, נחסום את $Pr[A_1 \geq \frac{\varepsilon}{2}]$. ובכן: אם קרה A_1 אזי חישובנו $\hat{a}$ שמקיים $\hat{a} < a$. ולכן: כיוון שהגדרנו את $\hat{a}$ להיות הנקודה המינימלית שקיבלנו בסט הנתונים שקיבלה את התיוג $y_i = 1$ וכן ראינו כי $Pr[x \in (a, a^*)] = \frac{\varepsilon}{2}$ אזי בהכרח נטען כי אף אחת מן הנקודות לא נפלה ב- (a, a^*) . שכן, אם נב"ש שישנה נקודה שנפלה שם, אנחנו יודעים כי ההסתברות לשגיאה זו תהיה קטנה מחצי אפסילון. (מדוע? כיוון שהשגיאה גדולה מחצי אפסילון, אך ליפול במרחק זה לכל היותר חצי אפסילון). לכן:

$$Pr[A_1 \geq \frac{\varepsilon}{2}] \leq Pr[\forall i \in [n] : x_i \notin (a, a^*)] \stackrel{iid}{=} \prod_{i=1}^n 1 - Pr[x_i \in (a, a^*)] =$$

$$\prod_{i=1}^n (1 - \frac{\varepsilon}{2}) = (1 - \frac{\varepsilon}{2})^n$$

כעת, בצורה דומה: נחסום את $Pr[A_2 \geq \frac{\varepsilon}{2}]$. אם מאורע 2 התרחש לפי הגדרת האלגוריתם הרי שישנו מקטע שאף אחת מהנקודות לא נפלה בו. באופן דומה להסבר הקודם, נקבל כי אף אחת מן הנקודות לא נפלה במקטע (b^*, b) : מכאן,

$$Pr[A_2 \geq \frac{\varepsilon}{2}] \leq Pr[\forall i \in [n] : x_i \notin (b^*, b)] =_{iid} \prod_{i=1}^n 1 - Pr[x_i \in (b^*, b)] =$$

$$\prod_{i=1}^n (1 - \frac{\varepsilon}{2}) = (1 - \frac{\varepsilon}{2})^n$$

לכן קיבלנו:

$$Pr[Err_{\mathcal{D}}(A(S)) > \varepsilon] \leq 2 \cdot (1 - \frac{\varepsilon}{2})^n \leq 2 \cdot e^{-\frac{n\varepsilon}{2}} \leq \delta$$

$$\Leftrightarrow e^{-\frac{n\varepsilon}{2}} \leq \frac{\delta}{2} \Leftrightarrow \frac{-n\varepsilon}{2} \leq \ln(\frac{\delta}{2}) \Leftrightarrow n \geq \frac{-2}{\varepsilon} \ln(\frac{\delta}{2}) = \frac{2}{\varepsilon} \ln(\frac{2}{\delta})$$

לכן, עבור $N(\delta, \varepsilon)$ המקיימים $n \geq \frac{2}{\varepsilon} \ln(\frac{2}{\delta})$ יתקיים $Pr[Err_{\mathcal{D}}(A(S)) > \varepsilon] \leq \delta$ ■
לכן, סה"כ $\mathcal{H}$ היא PAC לרנינג.

כללי

* טריק חשוב:

$$y_i = \beta^T x_i + \varepsilon_i \text{ s.t. } \varepsilon_i \sim \mathcal{N}(0, \sigma_i^2)$$

במקום לחשוב על זה כמשוואה חשבונית, נחשוב כיצד נוצר y_i . הוא נוצר ע"י ערך אמיני בתוספת טעות כלשהי. לכן, הסטיה היחידה שתחווה y_i היא בעקבות הרעש. גם חיסור גאוסיאנים גוסיאני והרי $\beta^T x_i$ קבוע. לכן:
כאן נשים לב. $\varepsilon_i = y_i - \beta^T x_i$. כעת, הרי שמדובר בקבוע שם לכן

$$y_i \sim \mathcal{N}(\beta^T x_i, \sigma_i^2)$$

* עבור כל וקטור z ומטריצה A מתקיים:

$$z^T A z = \sum_i \sum_j z_i A_{ij} z_j$$

* אלגברה לינארית:

בהינתן שרוצים לפתוח ביטוי מהצורה $\|\sum_{i=1}^n A_i\|_2^2$ אזי:

$$\|\sum_{i=1}^n A_i\|_2^2 = (\sum_{i=1}^n A_i)^T (\sum_{i=1}^n A_i) = \sum_{j=1}^n \sum_{i=1}^n A_i^T A_j$$

* איך מוכיחים כי מדגם נקודות פריד לינארית?

- יש למצוא וקטור מקדמים w כך שלכל נקודה מתקיים $y_i w^T x_i > 0$! זה התנאי! אגב - אם קיבלנו גדול מחצי, שליש, $\frac{1}{67}$ - זה מסווג שפריד לינארי!

- דוגמה חשובה: נתון כי המדגם $\{(x_i, y_i)\}_{i=1}^n$ מקיים כי הדגימות אורתונורמליות. אזי, הנתונים בוודאות פרידים לינאריים. **הוכחה:** נרצה להוכיח שקיים w כך שלכל נקודה מתקיים $y_i w^T x_i > 0$.

ובכן, הדגימות אורתונורמליות. לכן, קיים בסיס $B = \{x_1, \dots, x_n\}$ ל $B = \{x_1, \dots, x_n\}$ אורתונורמלי. יהי $w = \sum_{i=1}^n \alpha_i x_i$ כעת,

תהי נקודה x_i . נראה כי: $w^T x_i = \left(\sum_{j=1}^n \alpha_j x_j\right)^T x_i = \sum_{j=1}^n \alpha_j x_j^T x_i = \sum_{j=1}^n \alpha_j \langle x_j, x_i \rangle$. הרי שמדובר בבסיס אורתונורמלי. לכן, כל $j \neq i$ מתאפס בסכום זה. נקבל כי $w^T x_i = \alpha_i \langle x_i, x_i \rangle = \alpha_i$. הרי שמדובר בבסיס אורתונורמלי ולכן $\langle x_i, x_i \rangle = 1$. לכן, $w^T x_i = \alpha_i$. בעצם כעת עלינו להראות כי לכל נקודה יתקיים $y_i \alpha_i > 0$. נגדיר $\alpha_i = y_i$. כעת יתקיים אכן כי $y_i \alpha_i = y_i^2 > 0$ תמיד. לכן, עבור הוקטור

$$w = \sum_{i=1}^n y_i x_i$$

הנתונים פרידים לינאריים, כנדרש.

* זה לא בסיס המקורי אז נוסף: RNN - רשת ניורונים שבה יש חשיבות לסדר, למשל לטקסט, לסדרת מחירי מניות וכו'. בניגוד לרשת רגילה שמעבדת כל קלט בנפרד היא זוכרת מידע מהעבר באמצעות מצב פנימי. למשל, נניח שאנחנו קוראים משפט מילה אחר מילה - "החתול אכל את...". כדי לנחש את המילה הבאה (הגבינה) חשוב לזכור את כל המילים הקודמות. לכן בכל שלב היא מקבלת קלט נוכחי וזיכרון מצעד קודם ומחשבת זיכרון חדש. היא יכולה להתמודד כמובן עם קלט שבו רצפים באורכים שונים.

* טריק לתערובת גאוסיאנים (לא הועבר אצלנו הסמסטר, כן הועבר ב-2024): ניתן לכתוב את פונקציית העלות כ

$$P_\theta(x_1, \dots, x_n, z_1, \dots, z_n) = \prod_{i=1}^n \prod_{k=1}^K \pi_k^{Z_{i,k}} \mathcal{N}(x_i; \mu_k, \Sigma_k)^{z_{i,k}}$$

באשר $z_{i,k}$ אלו משתנים חבויים - 1 אם x_i משוייך ל- k . מדוע זה מגיע? כי הנוסחה המקורית היא

$$P_\theta(x_1, \dots, x_n) = \sum_{i=1}^K \log \left(\sum_{k=1}^K \Pr(x_i, z_i = k | \theta) \right)$$

זה Log של סכום, בעייתי. לכן ניתן להציג לעיל, ואז לגזור ולקבל:

$$\log \left(\prod_{i=1}^n \prod_{k=1}^K \pi_k^{Z_{i,k}} \mathcal{N}(x_i; \mu_k, \Sigma_k)^{z_{i,k}} \right) = \sum_{i=1}^n \sum_{k=1}^K Z_{i,k} \log(\pi_k) + Z_{i,k} \log(\mathcal{N}(x_i; \mu_k, \Sigma_k))$$

וזה כבר גזיר.

* כל האלגוריתמים $GD, Momentum, AdaGrad, RMSprop, Adam, AdamW$, פרט לאחרון ($AdamW$) הם שייכים למשפחת האופטימיזרים והמטרה המרכזית שלהם היא לשפר ולייעל את תהליך החיפוש וההתכנסות של המודל למינימום פונקציית העלות. האחרון גם משפר אוברפיטינג אך האחרים לא קשורים לכך.

* קשר לעצי החלטה: Boosting ($XGBOOST$) מתאמן על שאריות, ואילו $AdaBoost$ אלגוריתם שבו בכל שלב עוברים ומעדכנים את כולם אך אלו שטעינו בהם מקבלים משקל גבוה יותר בעדכון, על מנת שהמודל ידע "לשים עליהם עין".

* נשים לב שאם המדגם פריד לינארי ניתן להגדיר

$$\gamma = \min_i y_i w^T x_i$$

נגדיר מקדם c לכל w להיות $c =$ לכן לכל i :

$$c \cdot y_i w^T x_i \geq \frac{y_i w^T x_i}{\min_i y_i w^T x_i} \geq \frac{y_i w^T x_i}{y_i w^T x_i} = 1$$

ולכן יש הפרדה.